객체지향 프로그래밍 고급

5강 : 객체배열

객체 배열

- 객체가 여러 개인 경우 배열이나 포인터를 사용하는 것이 효율적이다.
  객체를 배열로 선언하는 것을 객체배열이라고 한다.

클래스명 객체배열명[배열크기];

```
Dog ar[3];
```

- 객체배열 선언은 자동으로 매개변수 없는 기본 생성자를 호출하므로 반드시 기본 생성자를 구현해야 한다.

```
Dog::Dog() { }
```

객체 배열 지정과 초기화

- 객체 배열을 지정하려면 일반 배열처럼 첨자(인덱스)를 사용하고, 각 멤버에 접근하려면 점(.)연산자를 붙인다.

```
객체배열명[첨자].멤버;
```

```
ar[0].showData();
ar[1].showData();
ar[2].showData();
```

- 객체 배열의 멤버를 초기화하려면 {} 속에 생성자를 사용한다.

```
클래스명 객체배열명[첨자] = {생성자() 또는 생성자(인자)
```

```
Dog ar[3] = { Dog(“Happy”, 3, 4), Dog(“marry”, 5, 6), Dog(“nuri”, 1, 2);
```

객체 배열 지정과 초기화(1)

```
class Point
{
private:
   int xpos;
   int ypos;
public:
   Point() { }
   Point(int x, int y);
   void Show();
};
Point::Point(int x, int y)
{
   xpos = x;
   ypos = y;
}
void Point::Show()
{
}
```

```
int main()
{
   Point arr[3] = { Point(100, 100),
                    Point(200, 200),
                    Point(300, 400) } ;

   for (int i=0; i<3; i++)
      arr[i].Show();

   return 0;
}
```

객체 배열 지정과 초기화(2)

```
class Point
{
private:
   int xpos;
   int ypos;
public:
   Point();
   Point(int x, int y);
   void Show() { . . . }
   void SetInfo(int, int);
};
Point::Point()
{     xpos = ypos = 0;    }

Point::Point(int x, int y)
{
   xpos = x;
   ypos = y;
}
```

```
void Point::SetInfo(int x, int y)
{
     xpos = x; ypos = y;
}

int main()
{
   Point arr[3];
   int x, y;
   for (int i=0; i<3; i++) {
       cin >> x, y;
       arr[i].SetInfo(x, y);
   }
   return 0;
}
```

[예제1] 도서관리

도서관리 클래스를 정의하고 아래 데이터를 출력하시오.

책명	저자	출판사	개수	가격
C++ 프로그램	이현창	한빛미디어	46	35,000
Java	박하대	정인당	25	11,000
HTML	설현	팩토리	24	9,900

[예제2]

아래 정보를 갖는 Student 클래스를 정의하고, 5명의 학생에 대해 총점과 평균점수를 구한 후, 총점이 높은 순으로 데이터를 출력하시오.

학번
이름
국어점수
수학점수
평균점수

학번	이름	국어	수학	총점	평균
2014	아이유	100	80		
2015	설현	90	97		
2016	김수현	79	88		
2017	장원영	80	100		
2019	김동원	93	93		

객체 배열과 객체 포인터

- 객체 포인터는 다음과 같이 객체 배열의 배열 요소를 참조할 수 있다.

```
클래스명  *객체명;                          클래스명  *객체명 = 객체배열명;
객체명 = 객체배열명;
```

```
Dog   ar[3];
Dog   *pt;
pt   =   ar;
```

```
Dog   ar[3];
Dog   *pt   =   ar;
```

- 객체 포인터로 객체 멤버에 점근할 때는 화살표(->) 연산자를 사용

```
객체포인터->멤버변수;
객체포인터->멤버함수(인자리스트);
```

```
pt->showData();
(pt+1)->showData();
```

객체 배열과 객체 포인터

```
Dog ar[3];
Dog *pt = ar;
```

```
class Simple
{
        int num;
Public:
        Simple(int n) {. . . };
        void show() { . . .}
}

int main()
{
        Simple ar[3] = { Simple(10), Simple(20), Simple(30)};
        Simple *pt = ar;
        pt->show();             //pt[0].show();
        (pt + 1)->show();
        (pt + 2)->show();
}
```

[예제3]

다음은 Dog 클래스이다. 멤버 함수를 정의하시오.
3개의 Dog 배열을 선언하고 아래의 값으로 초기화한 후, 배열이 아닌 포인터를 이용하여 멤버함수를 호출하시오.

이름	char[20]	happy	marry	nurry
나이	int	3	4	2
몸무게	int	4	5	7

[예제4]

X, Y 좌표값을 갖는 Point 클래스를 정의하라.
초기데이터:(10,10), (20,20), (30,30)을 갖는 Point 객체배열을 생성하고 데이터를 출력하시오.

배열의 동적 생성

```
class Circle
{
Private:
    Int radius;
Public:
    Circle() { radius = 1; }
    ~Circle() {}
    void setRadius(int r) { radius = r; }
    double getArea() {return 3.14 * radius *
    radius; }
};
```

```
int main()
{
    int n, radius;
    cin >> n;
    Circle *pArray = new Circle[n];
    for (int i=0;i<n;i++) {
        cout << "원의 반지름 : "; cin >> radius;
        pArray[i].setRadius(radius);
    }
    Circle *p = pArray;
    for (int i=0;i<n;i++) {
        cout << p->getArea() << endl;
        p++;
    }
    delete [] pArray;
}
```

[예제5]

이름과 전화번호를 갖는 Person 클래스를 정의하고, 아래와 같은 기능을 갖도록 구현하라. 데이터는 최대 10명을 넘지 않는다.

```
>> 추가는 1, 모두 보기는 2번을 누르시오.
>> 1
>> 이름 =>  홍길동
     전화번호 => 0101234567
    입력되었습니다.
>>  추가는 1, 수정은 2, 삭제는 3, 모두 보기는 4번을 누르시오.
>>
```

[예제]객체배열을 함수의 매개변수로 전달하기

```
class Dog
{
private:
	char name[20];
	unsigned int age;
	unsigned int weight;
public:
	Dog() { };
	Dog(char name[], int age, int weight);
	void showData();
};

Dog::Dog(char name[], int age, int weight)
{
	strncpy_s(this->name, name, 20);
	this->age = age;
	this->weight = weight;
}
```

```
void Dog::showData()
{
	cout << name << " " << age << " " << " " << weight << endl;
}
void showObj(Dog *pt)
{
	for (int i = 0; i < 3; i++)
		(pt + i)->showData();
}

int main()
{
	Dog ar[3] = { Dog("happy", 3, 4),
		Dog("marry", 4, 5), Dog("nurry", 2, 7)};
	Dog *par = ar;
	showObj(ar);
	return 0;
}
```

동적 할당과 객체 포인터 배열

- 객체 배열은 객체로 이루어진 배열.
- 객체 포인터 배열은 객체의 주소값 저장이 가능한 포인터 변수로 이루어진 배열.

```
SoSimple *ptrArr[3];
```

- 객체포인터배열과 동적할당

```
SoSimple *ptrArr[3];
ptrArr[0] = new SoSimple();
ptrArr[1] = new SoSimple();
ptrArr[2] = new SoSimple();
```

동적 할당과 객체 포인터 배열의 예

```
class Person
{
private:
	char *name;
	int age;
public:
	Person(char *n, int a)
	{
		int len = strlen(n) + 1;
		name = new char(len);
		strcpy(name, n);
		age = a;
	}
	Person()
	{
		name = NULL;
		age = 0;
	}
	void SetInfo(char *n, int a)
	{
		strcpy(name, n);
		age = a;
	}
	void ShowInfo()
	{
		cout << name << "  " << endl;
		cout << age << endl;
	}
	Person::~Person()
	{
		delete[] name;
	}
};
```

동적 할당과 객체 포인터 배열의 예

```
int main()
{
	Person *parr[3];
	char namestr[100];
	int age;

	for (int i = 0; i < 3; i++)
	{
		cin >> namestr;
		cin >> age;
		parr[i] = new Person(namestr, age);
		parr[i]->ShowInfo();
	}
	delete parr[0];
	delete parr[1];
	delete parr[2];
	return 0;
}
```

[예제6]

X, Y 좌표를 갖는 Point 클래스를 선언하고, 객체 포인터 배열과 동적할당을 통해 데이터를 저장한 후, 출력하시오.

Point* p[5];

데이터 :
(100,100)
(200,200)
(50,80)
(10,10)
(120,24)

```
Point* p[5];
p[0] = new Point(100, 100);
p[1] = new Point(100, 200);
P[2] = new Point(50, 80);
  .
  .
for (int i = 0; i < 5; i++)
      p[i]->showData();

for (int i = 0; i < 5; i++)
      delete p[i];
```

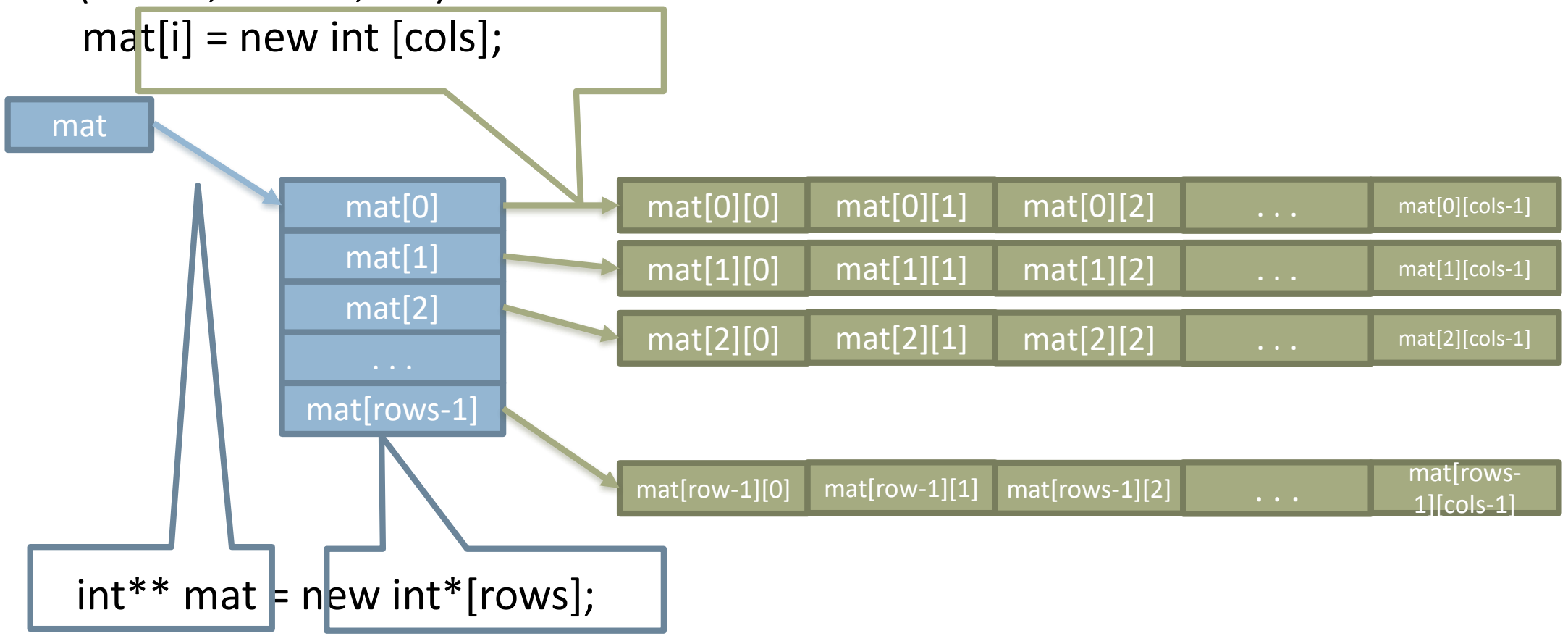
[예제7]

다음을 만족하는 클래스를 설계하라.

- Person 클래스는 이름(name), 나이(age), 핸드폰 번호(phone)를 속성으로 갖는다.
- 객체는 포인터 배열로 선언하고 동적할당한다.
- 데이터는 최대 10개를 넘지 않는다.
- 기능 : (1)추가 (2) 모두 보기

2차원 배열의 동적 할당

```
int** mat = new int*[rows];
for (int i=0; i<rows; i++)
      mat[i] = new int [cols];
```



2차원 배열의 메모리 해제

```
for (int i=0; i<rows; i++)
      delete [] mat[i];
delete [] mat;
```

[예제8] Matrix 클래스

2차원 배열의 동적할당을 이용하여 int값을 저장하는 행렬 클래스 Matrix를 설계하라.

[속성]
- 행(rows)
- 열(cols)
- 동적으로 할당된 메모리의 주소를 가리킬 포인터 변수(int** mat)

[함수]
- 생성자(행과 열의 크기를 인수로 받아 동적으로 행렬을 생성)
- 소멸자
- Print() : 화면에 행렬 내용을 출력
- setRand() : 행렬의 값을 랜덤숫자로 초기화(100보다 작은 정수)

[예제9] 주소록 서비스 만들기

이름, 회사, 핸드폰번호가 저장되는 주소록 서비스를 만들고자 한다. 주소록의 데이터는 파일에 저장한다.

최대 10개의 명함을 저장할 수 있다.

서비스의 기능

1. 주소록 검색(이름으로 검색)
2. 주소록 추가
3. 주소록 전체 보기

[과제 1]Banking System 1.0

- 은행에서 관리하는 은행계좌 관리프로그램을 구현하려고 한다. 기능은 다음과 같다.

1. **계좌개설**
2. **입　　금**
3. **출　　금**
4. **전체고객 잔액조회**

- 시스템에서 관리하는 고객의 계좌정보는 다음과 같다.

계좌번호, 고객이름, 주민번호, 예금 잔액
계좌번호는 신규 계좌가 생길 때마다 무작위 숫자(5자리)가 자동으로 부여된다.
고객의 식별키는 주민번호이다.

- Account 클래스를 설계하라.
- 데이터는 파일로 관리하라.
- 고객관리를 위해 객체 배열 포인터를 사용하고 고객이 추가될 때마다 동적 할당하라.
- 고객은 최대 100명을 넘지 않는다.

-----Menu ----
1. 계좌개설
2. 입금
3. 출금
4. 전체고객 잔액조회
5. 프로그램 종료
선택: 1

[계좌개설]
이름 : 홍길동
주민번호: 010308
입금액 : 10000
계좌개설이 완료되었습니다.

-----Menu ----
1. 계좌개설
2. 입금
3. 출금
4. 전체고객 잔액조회
5. 프로그램 종료
선택: 2

[입금]
주민번호: 010308
입금액 : 10000
입금이 완료되었습니다.
홍길동님의 잔액은 20000원입니다.

-----Menu ----
1. 계좌개설
2. 입금
3. 출금
4. 전체고객 잔액조회
5. 프로그램 종료
선택: 3

[계좌정보]
주민번호: 010308
이름 : 홍길동 (주민번호를 입력하면 이름 자동출력)
출금액 : 5000
출금이 완료되었습니다.
홍길동님의 잔액은 15000원입니다.

```
Class Account {
private:
	int accID;
	string regID;
	string name;
	int balance;
public:
	Account(int id, int money, string rid, string n) :accID(id), balance(money) {
		regID = rid;
		name = n;
	}
	string GetRegID() {return rid};
	void Deposit(int money) { balance += money; }  // 입금
	int Withdraw(int money)  // 출금
	void ShowAll();
	~Account();
};
Account *arr[100];
int accNum = 0;
```

[예제5]몬스터 사냥 게임

게임이 시작되면 사용자는 **생성할 몬스터의 수**를 입력한다.
프로그램은 입력된 수만큼 **동적으로 몬스터 객체를 생성**하여 게임에 배치한다.
각 몬스터는 다음 세 종류 중 하나를 선택하여 생성한다.

번호	몬스터	HP
1	슬라임	50
2	오크	100
3	드래곤	300

`Monster` 클래스에는 다음 멤버를 포함한다.

- 몬스터 이름(name)
- 체력(hp)

다음 기능을 구현한다.

- 기본 생성자
- 생성자 오버로딩
- `Print()` : 몬스터 정보를 출력

프로그램 실행 순서

1.생성할 몬스터 수를 입력받는다.

2.몬스터 객체를 저장할 **객체 포인터 배열을 동적으로 생성**한다.

3.각 몬스터의 종류를 입력받아 해당 객체를 생성한다.

4.생성된 모든 몬스터의 정보를 출력한다.

5.프로그램 종료 전에 생성한 모든 객체와 배열의 메모리를 해제한다.

```
class Monster { private: string name; int hp; public: Monster() { name = "슬라임"; hp = 50; } Monster(string n, int h) { name = n; hp = h; } void Print() { cout << "몬스터 : " << name << " HP : " << hp << endl; } string GetName() { return name; } int GetHP() { return hp; } }; int main() { int count; cout << "생성할 몬스터 수 : "; cin >> count; // 객체 포인터 배열 동적 생성 Monster** monsters = new Monster*[count]; for (int i = 0; i < count; i++) { int type; cout << endl; cout << i + 1 << "번째 몬스터 선택" << endl; cout << "1. 슬라임 2. 오크 3. 드래곤 : "; cin >> type; switch (type) { case 1: monsters[i] = new Monster("슬라임", 50); break; case 2: monsters[i] = new Monster("오크", 100); break; case 3: monsters[i] = new Monster("드래곤", 300); break; default: monsters[i] = new Monster(); } } cout << "\n===== 생성된 몬스터 =====\n"; for (int i = 0; i < count; i++) { monsters[i]->Print(); } // 메모리 해제 for (int i = 0; i < count; i++) { delete monsters[i]; } delete[] monsters; return 0; }
```

[예제5-2]

프로그램을 수정하여 플레이어가 몬스터를 공격하도록 구현하시오.

- 공격할 몬스터 번호를 입력받는다.
- 공격력이 30이라고 가정하고 몬스터의 HP를 감소시킨다.
- HP가 0 이하가 되면 "○○ 처치!"를 출력한다.
- 처치된 몬스터는 `delete`로 메모리를 해제하고 포인터를 `nullptr`로 설정한다.